0020. Valid Parentheses

y: Easy | Category: Stack | LeetCode: https://leetcode.com/problems/valid-paren

Problem

- S contains only ()[]{}. It is valid if every opener is closed by the same type in the correct order.
- Example: "()[]{}" → true, "(]" → false, "([)]" → false.

Intuition

- The most recent unclosed opener must be closed first: last in, first out, so use a stack.
- Trick: push the expected closer instead of the opener, then a closer just has to equal the popped value.

Brute Force

- Repeatedly delete "()", "[]", "{}" until nothing changes: O(n^2).

Optimal Approach

- For each char: if it is an opener, push its matching closer.
- Else if the stack is empty or pop() != char, return false.
- At the end, return stack.isEmpty().

Worked Example

```
s = "([)]".

'(' push ')'   stack=[)]
'[' push ']'   stack=[),]]
')' pop ']'    ']' != ')' -> false
```

Java Solution

```java
public class ValidParentheses {
    public boolean isValid(String s) {
        Deque<Character> st = new ArrayDeque<>();
        for (char c : s.toCharArray()) {
            if (c == '(') st.push(')');
            else if (c == '[') st.push(']');
            else if (c == '{') st.push('}');
            else if (st.isEmpty() || st.pop() != c) return false;
        }
        return st.isEmpty();
    }
}
```

Complexity

- Time: O(n). Space: O(n).

Pitfalls

- Leftover openers ("((") mean invalid, so check emptiness at the end.
- A closer on an empty stack ("]") is invalid.